```
!pip install folium geopandas scikit-learn xgboost matplotlib seaborn plotly -q
print("✅ All dependencies installed.")
```

```
✅ All dependencies installed.
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import folium
from folium.plugins import HeatMap, MarkerCluster
import warnings
warnings.filterwarnings("ignore")

plt.rcParams["figure.dpi"] = 130
sns.set_theme(style="darkgrid")
print("✅ All libraries loaded.")
```

```
✅ All libraries loaded.
```

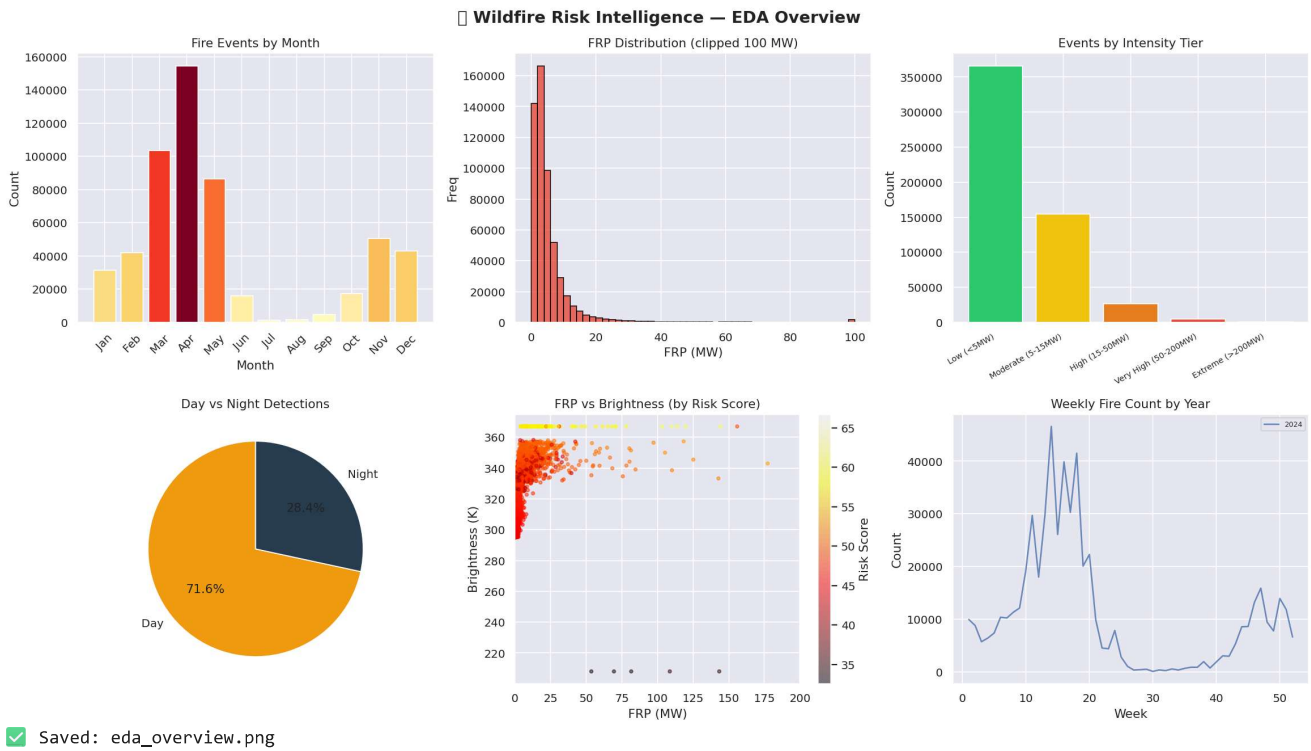
```
from google.colab import files
uploaded = files.upload()
df = pd.read_csv(list(uploaded.keys())[0])

print(f"📦 Shape       : {df.shape}")
print(f"\n📋 Columns:\n{df.dtypes}")
print(f"\n📊 Summary:\n{df.describe()}")
print(f"\n🔍 Missing:\n{df.isnull().sum()}")
print(f"\n👀 First 5 rows:\n{df.head()}")
```

```
# Parse datetime
df["acq_date"]  = pd.to_datetime(df["acq_date"])
df["year"]      = df["acq_date"].dt.year
df["month"]     = df["acq_date"].dt.month
df["dayofyear"] = df["acq_date"].dt.dayofyear
df["week"]      = df["acq_date"].dt.isocalendar().week.astype(int)

# VIIRS confidence l/n/h → numeric
df["conf_numeric"] = df["confidence"].map({"l": 1, "n": 2, "h": 3}).fillna(2)

# FRP intensity tiers
df["frp_tier"] = pd.cut(
    df["frp"],
    bins=[0, 5, 15, 50, 200, np.inf],
    labels=["Low (<5MW)", "Moderate (5-15MW)", "High (15-50MW)",
            "Very High (50-200MW)", "Extreme (>200MW)"]
)

# Day / Night label
df["period"] = df["daynight"].map({"D": "Day", "N": "Night"})

# 0.5° spatial grid
df["lat_grid"] = (df["latitude"]  // 0.5) * 0.5
df["lon_grid"] = (df["longitude"] // 0.5) * 0.5
df["grid_id"]  = df["lat_grid"].astype(str) + "_" + df["lon_grid"].astype(str)

# Composite risk score (0-100)
df["risk_score"] = (
    0.4 * (df["frp"]          / df["frp"].max()          * 100) +
    0.3 * (df["brightness"]   / df["brightness"].max()   * 100) +
    0.2 * (df["conf_numeric"] / 3                        * 100) +
    0.1 * (df["bright_t31"]   / df["bright_t31"].max()   * 100)
)

print("✅ Feature engineering done.")
print(df[["acq_date","frp","frp_tier","risk_score","conf_numeric"]].head(10))
```

```
75%          0.540000    1932.000000       2.0     367.800000       8.210000
max          6.480000    2159.000000       2.0     423.340000    1552.820000
✅ Feature engineering done.
    acq_date   frp          frp_tier  risk_score  conf_numeric
0 2024-01-01  1.89         Low (<5MW)   47.481907             2
1 2024-01-01  1.06         Low (<5MW)   47.616907             2
2 2024-01-01  1.75         Low (<5MW)   47.664765             2
3 2024-01-01  1.37         Low (<5MW)   47.520875             2
4 2024-01-01  2.07         Low (<5MW)   47.252310             2
5 2024-01-01  2.05         Low (<5MW)   40.704481             1
6 2024-01-01  1.68         Low (<5MW)   47.528709             2
7 2024-01-01  1.89         Low (<5MW)   47.189655             2
8 2024-01-01  5.04    Moderate (5-15MW)   57.216477             3
9 2024-01-01  2.04         Low (<5MW)   47.347986             2
🔍 Missing:
```

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
fig.suptitle("🔥 Wildfire Risk Intelligence — EDA Overview", fontsize=16, fontweight="bold")

# Monthly fire count
monthly = df.groupby("month").size().reset_index(name="count")
axes[0,0].bar(monthly["month"], monthly["count"],
              color=plt.cm.YlOrRd(monthly["count"] / monthly["count"].max()))
axes[0,0].set(title="Fire Events by Month", xlabel="Month", ylabel="Count")
axes[0,0].set_xticks(range(1,13))
axes[0,0].set_xticklabels(["Jan","Feb","Mar","Apr","May","Jun",
                           "Jul","Aug","Sep","Oct","Nov","Dec"], rotation=45)

# FRP distribution
axes[0,1].hist(df["frp"].clip(0,100), bins=50, color="#e74c3c", edgecolor="black", alpha=0.8)
axes[0,1].set(title="FRP Distribution (clipped 100 MW)", xlabel="FRP (MW)", ylabel="Freq")

# Tier counts
tier_counts = df["frp_tier"].value_counts().sort_index()
colors = ["#2ecc71","#f1c40f","#e67e22","#e74c3c","#8e44ad"]
axes[0,2].bar(range(len(tier_counts)), tier_counts.values, color=colors)
axes[0,2].set_xticks(range(len(tier_counts)))
axes[0,2].set_xticklabels(tier_counts.index, rotation=30, ha="right", fontsize=8)
axes[0,2].set(title="Events by Intensity Tier", ylabel="Count")

# Day vs Night
period_counts = df["period"].value_counts()
```

```
axes[1,0].pie(period_counts.values, labels=period_counts.index,
              colors=["#f39c12","#2c3e50"], autopct="%1.1f%%", startangle=90)
axes[1,0].set_title("Day vs Night Detections")

# Brightness vs FRP
sample = df.sample(5000, random_state=42)
sc = axes[1,1].scatter(sample["frp"], sample["brightness"],
                       c=sample["risk_score"], cmap="hot", alpha=0.5, s=10)
plt.colorbar(sc, ax=axes[1,1], label="Risk Score")
axes[1,1].set(title="FRP vs Brightness (by Risk Score)",
              xlabel="FRP (MW)", ylabel="Brightness (K)")
axes[1,1].set_xlim(0, 200)

# Weekly trend
weekly = df.groupby(["year","week"]).size().reset_index(name="count")
for yr, grp in weekly.groupby("year"):
    axes[1,2].plot(grp["week"], grp["count"], label=str(yr), alpha=0.8)
axes[1,2].set(title="Weekly Fire Count by Year", xlabel="Week", ylabel="Count")
axes[1,2].legend(fontsize=7)

plt.tight_layout()
plt.savefig("eda_overview.png", bbox_inches="tight")
plt.show()
print("✅ Saved: eda_overview.png")
```



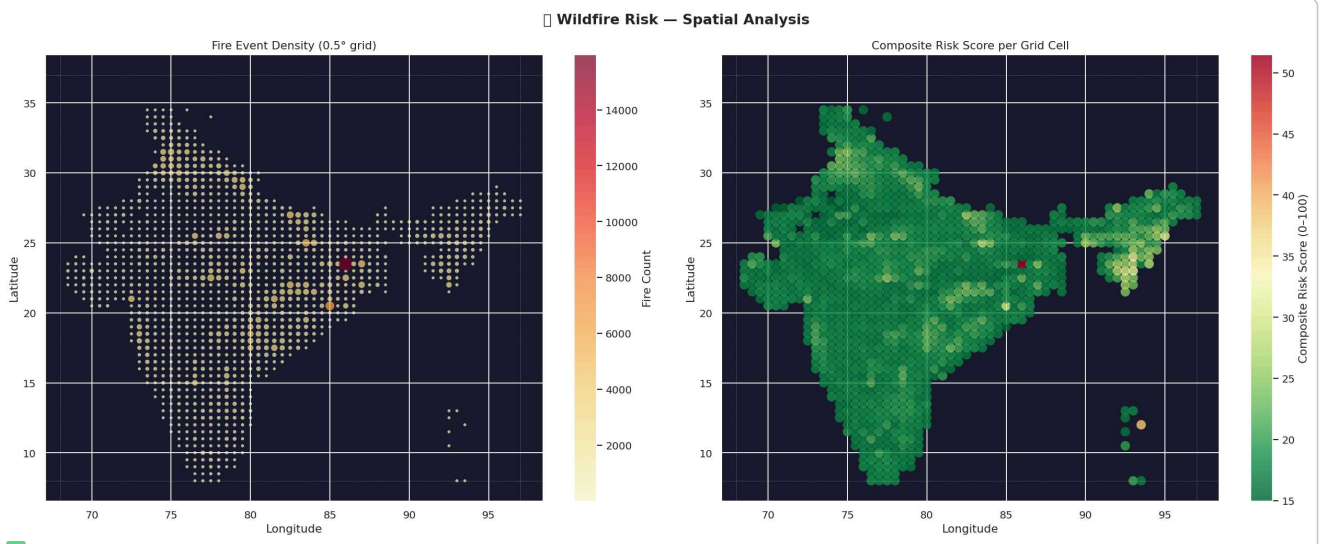
✅ Saved: eda_overview.png

```
grid = df.groupby(["lat_grid","lon_grid"]).agg(
    fire_count    = ("frp",          "count"),
    mean_frp      = ("frp",          "mean"),
    max_frp       = ("frp",          "max"),
    mean_bright   = ("brightness",   "mean"),
    mean_risk     = ("risk_score",   "mean"),
    high_conf_pct = ("conf_numeric", lambda x: (x == 3).mean() * 100)
).reset_index()

grid["norm_count"] = grid["fire_count"] / grid["fire_count"].max() * 100
```

```
grid["composite_risk"] = (
    0.35 * grid["norm_count"] +
    0.35 * grid["mean_risk"] +
    0.20 * (grid["mean_frp"] / grid["mean_frp"].max() * 100) +
    0.10 * grid["high_conf_pct"]
)

grid["risk_zone"] = pd.cut(
    grid["composite_risk"],
    bins=[0, 20, 40, 60, 80, 100],
    labels=["Very Low","Low","Moderate","High","Extreme"]
)

print(f"🗺️  Grid cells: {len(grid)}")
print("\nRisk Zone Distribution:")
print(grid["risk_zone"].value_counts().sort_index())
print("\nTop 10 High-Risk Cells:")
print(grid.nlargest(10, "composite_risk")[
    ["lat_grid","lon_grid","fire_count","mean_frp","composite_risk","risk_zone"]
].to_string(index=False))
```

```
🗺️  Grid cells: 1194

Risk Zone Distribution:
risk_zone
Very Low    1009
Low          184
Moderate       1
High           0
Extreme        0
Name: count, dtype: int64

Top 10 High-Risk Cells:
 lat_grid  lon_grid  fire_count   mean_frp  composite_risk risk_zone
     23.5      86.0       15974   2.680100       51.485238  Moderate
     12.0      93.5          17 106.571765       38.214042       Low
     23.0      93.0         760  61.374776       31.211171       Low
     24.0      93.0        1079  53.811520       30.981489       Low
     25.5      95.0           9  51.524444       30.662405       Low
     23.0      92.5        1392  45.233182       29.990619       Low
     20.5      85.0        6131   2.642567       29.899357       Low
     23.5      93.0         689  44.027504       28.272606       Low
     22.5      92.5        1465  34.092601       27.807445       Low
     25.0      83.5        4374   5.687833       27.548512       Low
```

```
fig, axes = plt.subplots(1, 2, figsize=(20, 8))
fig.suptitle("🔥 Wildfire Risk — Spatial Analysis", fontsize=15, fontweight="bold")

sc1 = axes[0].scatter(grid["lon_grid"], grid["lat_grid"],
                      c=grid["fire_count"], cmap="YlOrRd",
                      s=grid["fire_count"]/grid["fire_count"].max()*200+10,
                      alpha=0.7, edgecolors="none")
plt.colorbar(sc1, ax=axes[0], label="Fire Count")
axes[0].set(title="Fire Event Density (0.5° grid)",
            xlabel="Longitude", ylabel="Latitude")
axes[0].set_facecolor("#1a1a2e")

sc2 = axes[1].scatter(grid["lon_grid"], grid["lat_grid"],
                      c=grid["composite_risk"], cmap="RdYlGn_r",
                      s=100, alpha=0.8, edgecolors="none")
plt.colorbar(sc2, ax=axes[1], label="Composite Risk Score (0–100)")
axes[1].set(title="Composite Risk Score per Grid Cell",
            xlabel="Longitude", ylabel="Latitude")
axes[1].set_facecolor("#1a1a2e")

for ax in axes:
    for y in [8, 37]: ax.axhline(y=y, color="white", lw=0.4, ls="--", alpha=0.4)
    for x in [68, 97]: ax.axvline(x=x, color="white", lw=0.4, ls="--", alpha=0.4)

plt.tight_layout()
plt.savefig("spatial_risk_map.png", bbox_inches="tight", facecolor="#1a1a2e")
plt.show()
print("✅ Saved: spatial_risk_map.png")
```



✅ Saved: spatial_risk_map.png

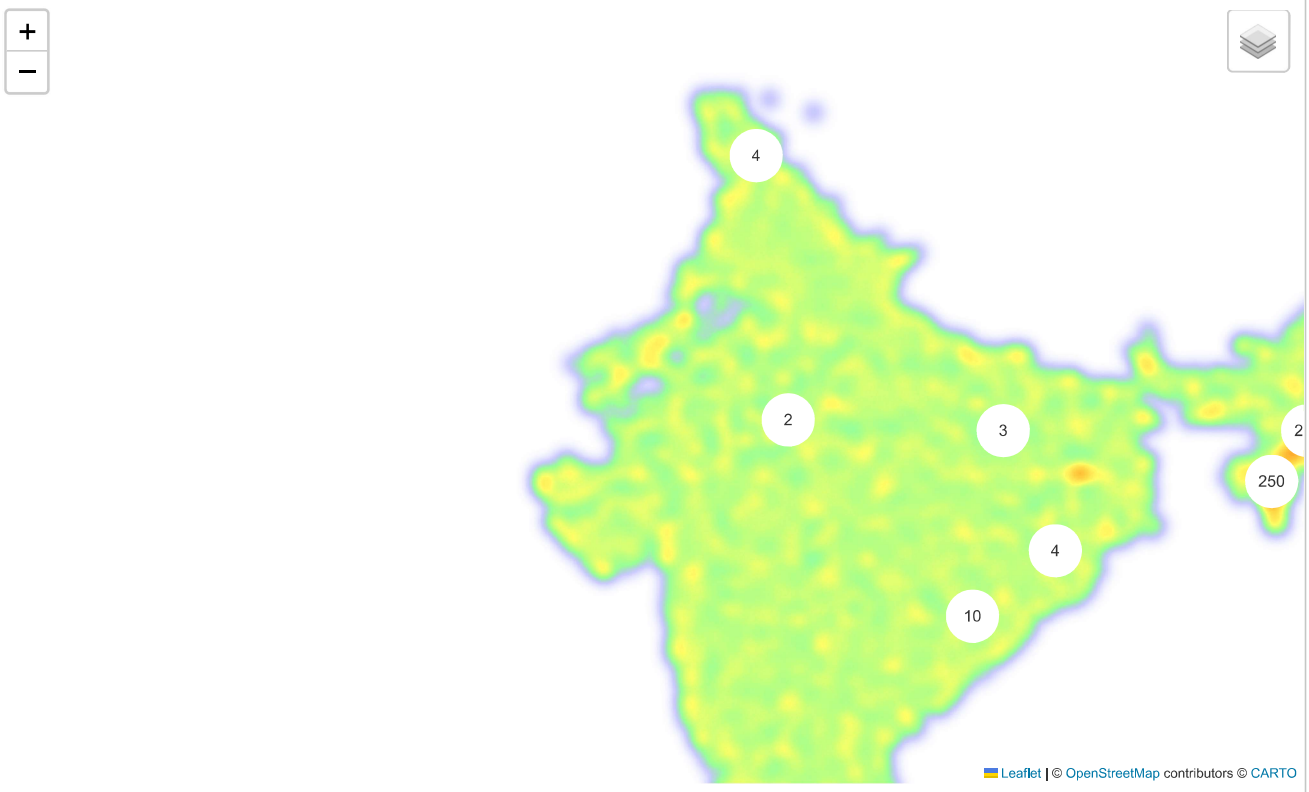
```
fig = px.scatter_mapbox(
    grid,
    lat="lat_grid", lon="lon_grid",
    color="composite_risk",
    size="fire_count",
    size_max=25,
    color_continuous_scale="RdYlGn_r",
    range_color=[0, 100],
    hover_data={"fire_count": True, "mean_frp": ":.2f",
                "max_frp": ":.1f", "composite_risk": ":.1f", "risk_zone": True},
    labels={"composite_risk":"Risk Score","fire_count":"Fire Count",
            "mean_frp":"Avg FRP (MW)","max_frp":"Max FRP (MW)"},
    mapbox_style="carto-darkmatter",
    zoom=4, center={"lat": 22, "lon": 80},
    title="🔥 Wildfire Risk Intelligence Engine — Interactive Map",
    height=700
)
fig.update_layout(margin={"r":0,"t":50,"l":0,"b":0})
fig.write_html("interactive_risk_map.html")
fig.show()
print("✅ Saved: interactive_risk_map.html")
```

🔥 Wildfire Risk Intelligence Engine — Interactive Map

✅ Saved: interactive_risk_map.html

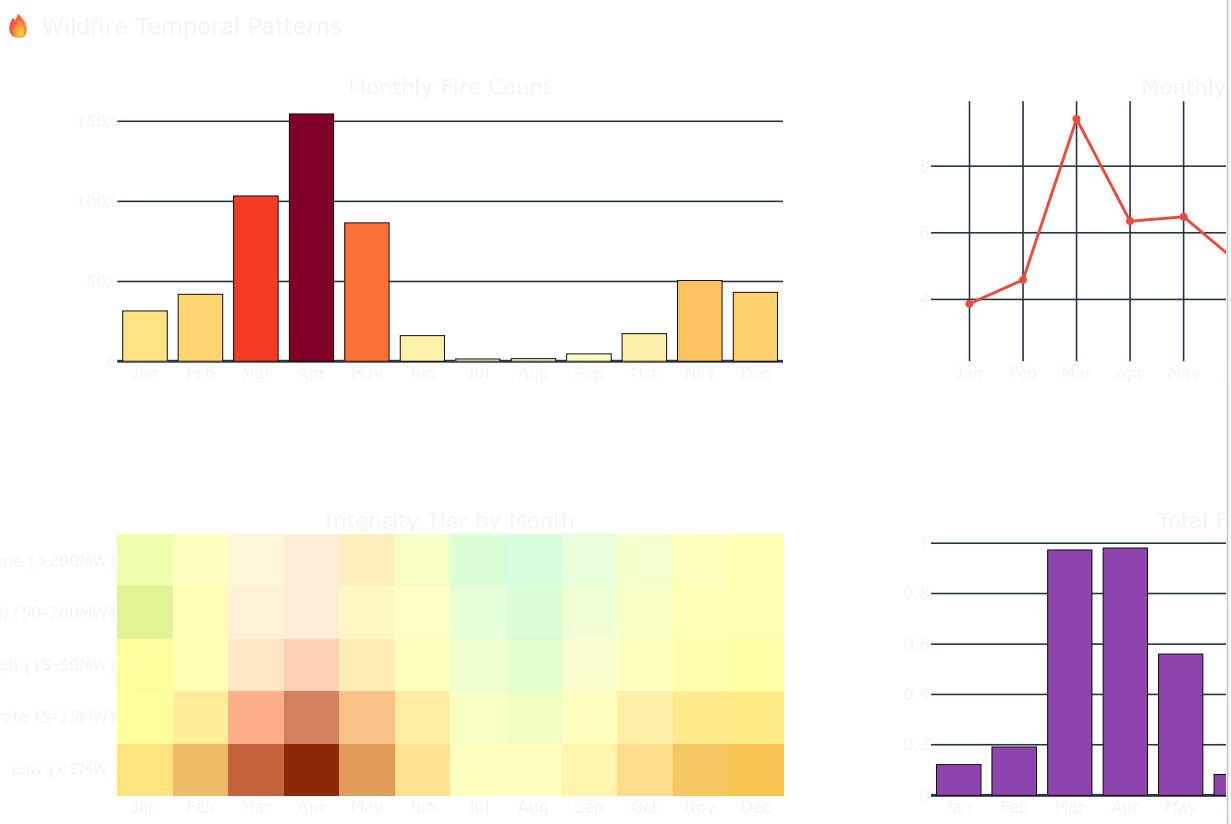
```
m = folium.Map(location=[22, 80], zoom_start=5, tiles="CartoDB dark_matter")

heat_data = df[["latitude","longitude","frp"]].dropna().values.tolist()
HeatMap(heat_data, min_opacity=0.3, radius=8, blur=10, max_zoom=7,
        gradient={0.2:"blue", 0.4:"lime", 0.6:"yellow",
                  0.8:"orange", 1.0:"red"}).add_to(m)

extreme = df[df["frp"] >= 200].copy()
mc = MarkerCluster(name="Extreme Fires (FRP ≥ 200 MW)").add_to(m)
for _, row in extreme.head(500).iterrows():
    folium.CircleMarker(
        location=[row["latitude"], row["longitude"]],
        radius=6, color="#ff0000", fill=True,
        fill_color="#ff4444", fill_opacity=0.8,
        popup=folium.Popup(
            f"<b>🔥 Extreme Fire</b><br>Date: {row['acq_date'].date()}<br>"
            f"FRP: {row['frp']:.1f} MW<br>Brightness: {row['brightness']:.1f} K<br>"
            f"Risk Score: {row['risk_score']:.1f}", max_width=200)
    ).add_to(mc)

folium.LayerControl().add_to(m)
m.save("wildfire_heatmap_folium.html")
print(f"✅ Saved: wildfire_heatmap_folium.html  |  Extreme fires: {len(extreme)}")
m
```

```
Saved: wildfire_heatmap_folium.html  |  Extreme fires: 483
```




```
months_label = ["Jan","Feb","Mar","Apr","May","Jun",
                "Jul","Aug","Sep","Oct","Nov","Dec"]

monthly_full = df.groupby("month").agg(
    count    =("frp","count"),
    avg_frp  =("frp","mean"),
    total_frp=("frp","sum")
).reset_index()

fig = make_subplots(rows=2, cols=2,
    subplot_titles=["Monthly Fire Count","Monthly Avg FRP (MW)",
                    "Intensity Tier by Month","Total FRP per Month"])

fig.add_trace(go.Bar(x=months_label, y=monthly_full["count"],
    marker=dict(color=monthly_full["count"], colorscale="YlOrRd")), row=1, col=1)

fig.add_trace(go.Scatter(x=months_label, y=monthly_full["avg_frp"],
    mode="lines+markers", line=dict(color="#e74c3c", width=2)), row=1, col=2)

tier_order = ["Low (<5MW)","Moderate (5-15MW)","High (15-50MW)",
              "Very High (50-200MW)","Extreme (>200MW)"]
tier_month = (df.groupby(["month","frp_tier"]).size()
               .unstack(fill_value=0)
               .reindex(columns=tier_order, fill_value=0))
fig.add_trace(go.Heatmap(z=tier_month.values.T,
    x=[months_label[i-1] for i in tier_month.index],
    y=tier_order, colorscale="YlOrRd", showscale=False), row=2, col=1)

fig.add_trace(go.Bar(x=months_label, y=monthly_full["total_frp"]/1e6,
    marker_color="#8e44ad"), row=2, col=2)

fig.update_layout(title_text="🔥 Wildfire Temporal Patterns",
```

```
                  height=700, showlegend=False, template="plotly_dark")
fig.write_html("temporal_analysis.html")
fig.show()
print("✅ Saved: temporal_analysis.html")
```




```
✅ Saved: temporal_analysis.html
```

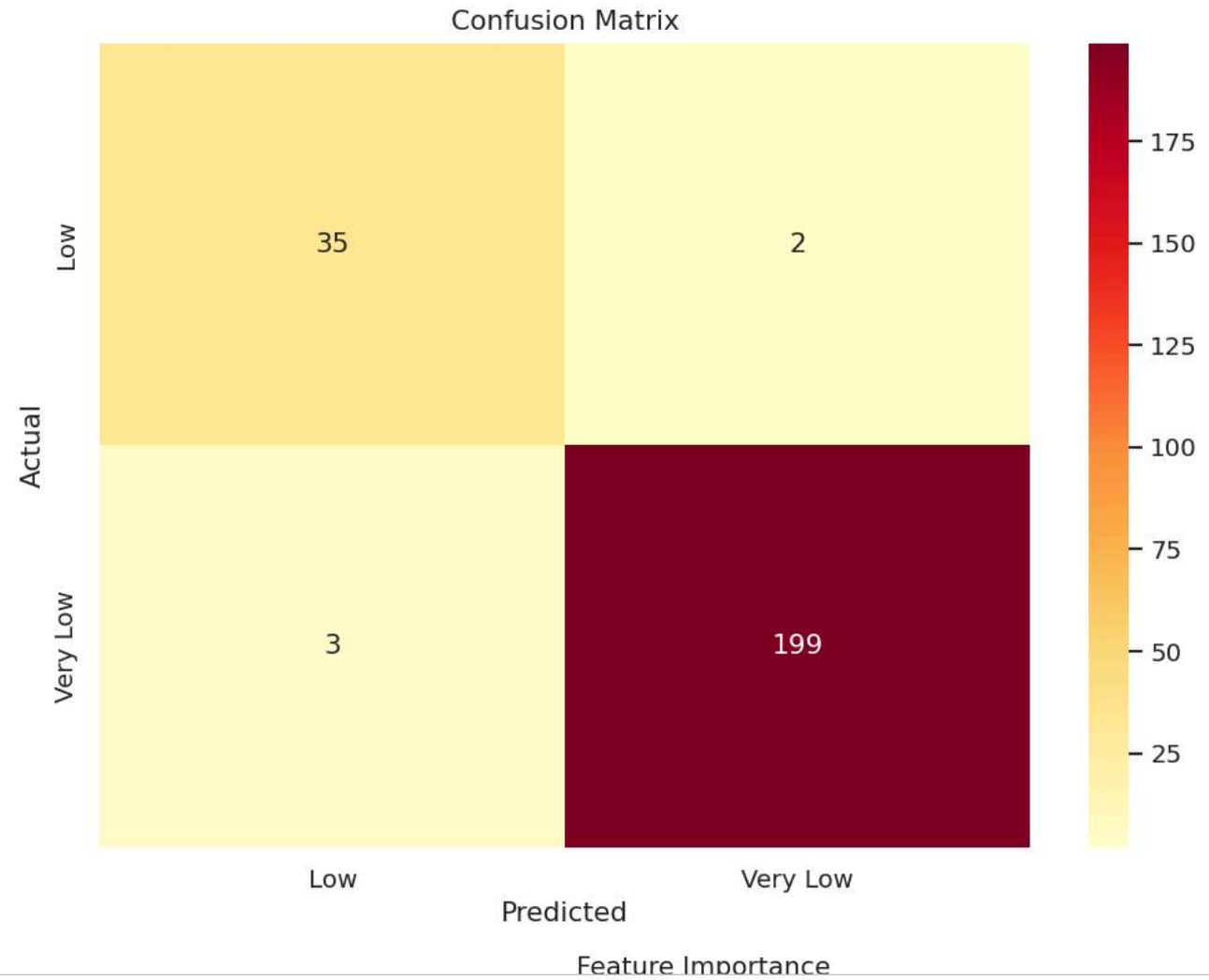
```
print("\n" + "="*60)
print("🔥 WILDFIRE RISK INTELLIGENCE — TOP PRONE REGIONS")
print("="*60)

extreme_grid = grid[grid["risk_zone"].isin(["Extreme","High"])].copy()
extreme_grid = extreme_grid.sort_values("composite_risk", ascending=False)
print(f"\n⚠️  High/Extreme cells: {len(extreme_grid)} ({len(extreme_grid)/len(grid)*100:.1f}% of all)")

for zone, grp in extreme_grid.groupby("risk_zone", observed=True):
    print(f"\n{'='*50}\n 🔴 ZONE: {zone}  ({len(grp)} cells)\n{'='*50}")
    print(f"  {'Lat':>8} {'Lon':>8} {'Count':>8} {'AvgFRP':>8} {'MaxFRP':>8} {'Risk':>8}")
    for _, row in grp.head(10).iterrows():
        print(f"  {row['lat_grid']:>8.2f} {row['lon_grid']:>8.2f} "
              f"{int(row['fire_count']):>8} {row['mean_frp']:>8.1f} "
              f"{row['max_frp']:>8.1f} {row['composite_risk']:>8.1f}")

print("\n📊 RISK ZONE SUMMARY")
print("-"*60)
summary = grid.groupby("risk_zone", observed=True).agg(
    cells=("lat_grid","count"), total_fires=("fire_count","sum"),
    avg_frp=("mean_frp","mean"), max_frp=("max_frp","max"),
    avg_risk=("composite_risk","mean")
).reset_index()
print(summary.to_string(index=False))
```

```
============================================================
🔥 WILDFIRE RISK INTELLIGENCE — TOP PRONE REGIONS
============================================================

⚠️  High/Extreme cells: 0 (0.0% of all)

📊 RISK ZONE SUMMARY
```

```
------------------------------------------------------------
risk_zone  cells  total_fires   avg_frp  max_frp  avg_risk
 Very Low   1009       286292  4.950742   365.74 18.025213
      Low    184       250046 12.540206  1552.82 22.490934
 Moderate      1        15974  2.680100    24.46 51.485238
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix
import xgboost as xgb
import numpy as np

ml_df = grid[["fire_count","mean_frp","max_frp","mean_bright",
              "high_conf_pct","norm_count","risk_zone"]].dropna().copy()

# Fix: Combine 'Moderate' risk zone with 'Low' risk zone due to 'Moderate' having only 1 sample,
# which causes issues for stratified train_test_split.
ml_df["risk_zone"] = ml_df["risk_zone"].replace("Moderate", "Low")

le = LabelEncoder()
ml_df["risk_label"] = le.fit_transform(ml_df["risk_zone"])

FEATURES = ["fire_count","mean_frp","max_frp","mean_bright","high_conf_pct","norm_count"]
X = ml_df[FEATURES]
y = ml_df["risk_label"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Fix: Explicitly set num_class based on the number of unique labels.
# Fix: Explicitly set objective to 'multi:softprob' for mlogloss compatibility.
model = xgb.XGBClassifier(n_estimators=200, max_depth=5, learning_rate=0.1,
                          objective='multi:softprob', num_class=len(le.classes_),
                          use_label_encoder=False, eval_metric="mlogloss", random_state=42)
model.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)

# Fix: Convert predicted probabilities to class labels for classification_report
y_pred = model.predict(X_test)
y_pred_labels = np.argmax(y_pred, axis=1)

print("📈 XGBoost — Classification Report")
print("="*55)
print(classification_report(y_test, y_pred_labels, target_names=le.classes_, zero_division=0))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_labels)
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="YlOrRd",
            xticklabels=le.classes_, yticklabels=le.classes_, ax=ax)
ax.set(title="Confusion Matrix", xlabel="Predicted", ylabel="Actual")
plt.tight_layout(); plt.savefig("confusion_matrix.png"); plt.show()

# Feature importance
feat_imp = pd.DataFrame({"Feature": FEATURES,
                         "Importance": model.feature_importances_}
                        ).sort_values("Importance", ascending=False)
fig, ax = plt.subplots(figsize=(8, 4))
ax.barh(feat_imp["Feature"], feat_imp["Importance"], color="#e74c3c")
ax.set(title="Feature Importance", xlabel="Score")
plt.tight_layout(); plt.savefig("feature_importance.png"); plt.show()
print("✅ Model done. Saved: confusion_matrix.png, feature_importance.png")
```

```
📈 XGBoost — Classification Report
=====================================================
              precision    recall  f1-score   support

         Low       0.92      0.95      0.93        37
    Very Low       0.99      0.99      0.99       202

    accuracy                           0.98       239
   macro avg       0.96      0.97      0.96       239
weighted avg       0.98      0.98      0.98       239
```



Feature Importance

```
import numpy as np

def predict_risk(fire_count, mean_frp, max_frp, mean_bright,
                 high_conf_pct, norm_count=None):
    if norm_count is None:
        norm_count = fire_count / grid["fire_count"].max() * 100
    features = pd.DataFrame(
        [[fire_count, mean_frp, max_frp, mean_bright, high_conf_pct, norm_count]],
        columns=["fire_count","mean_frp","max_frp","mean_bright","high_conf_pct","norm_count"]
    )

    # model.predict returns probabilities when objective='multi:softprob'
    proba = model.predict(features)[0]
    pred_label = np.argmax(proba) # Get the index of the class with highest probability
    zone  = le.inverse_transform([pred_label])[0] # Inverse transform the single predicted label

    print(f"\n{'='*45}\n  🔥 Predicted Zone : {zone}\n  Confidence       : {proba[pred_label]*100:.1f}%")
    print("\n  Class probabilities:")
    for cls, p in zip(le.classes_, proba):
        print(f"    {cls:<20} {p*100:>5.1f}%  {'▉'*int(p*30)}")
    print("="*45)
    return zone

# --- Try your own values below ---
predict_risk(fire_count=850, mean_frp=22.5, max_frp=180.0,
```

```
              mean_bright=345.0, high_conf_pct=60.0)

predict_risk(fire_count=12, mean_frp=3.2, max_frp=8.0,
              mean_bright=315.0, high_conf_pct=10.0)
```

```
==================================================
  🔥 Predicted Zone : Low
  Confidence       : 100.0%

  Class probabilities:
    Low                  100.0%  ████████████████████████████
    Very Low               0.0%
==================================================

==================================================
  🔥 Predicted Zone : Very Low
  Confidence       : 100.0%

  Class probabilities:
    Low                    0.0%
    Very Low             100.0%  ████████████████████████████
==================================================
'Very Low'
```

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

feat_cl  = df[["latitude","longitude","frp","brightness","risk_score"]].dropna().copy()
X_scaled = StandardScaler().fit_transform(feat_cl)

# Elbow
inertias = [KMeans(n_clusters=k, random_state=42, n_init=10).fit(X_scaled).inertia_
            for k in range(2, 11)]

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
axes[0].plot(range(2,11), inertias, "bo-", lw=2)
axes[0].axvline(x=5, color="red", ls="--", label="K=5")
axes[0].set(title="Elbow Method", xlabel="K", ylabel="Inertia"); axes[0].legend()

# Fit K=5
km = KMeans(n_clusters=5, random_state=42, n_init=10)
feat_cl["cluster"] = km.fit_predict(X_scaled)

rank   = feat_cl.groupby("cluster")["risk_score"].mean().sort_values(ascending=False).index
```